

Segment -3

Dynamic Programming (DP)

Dynamic programming is a method for solving complex problems by breaking them down into simpler sub-problems. It is applicable to problems that exhibit **overlapping sub-problems** and **optimal substructure**.

Key Elements of Dynamic Programming

1. **Optimal Substructure:**
 - A problem exhibits optimal substructure if an optimal solution can be constructed efficiently from optimal solutions of its subproblems. This means that the solution to the overall problem can be derived from solutions to its smaller subproblems.
2. **Overlapping Subproblems:**
 - This occurs when a problem can be broken down into smaller, recurring subproblems that are solved multiple times. Instead of solving the same subproblem repeatedly, dynamic programming solves each subproblem once and stores the result.

Approaches to Dynamic Programming

1. **Top-Down Approach (Memoization):**
 - In this approach, the problem is solved recursively, but results of subproblems are stored (memoized) to avoid redundant calculations. This is useful for problems that can be expressed recursively.
2. **Bottom-Up Approach (Tabulation):**
 - This approach involves solving all possible subproblems and storing their results in a table. The final solution is built up from these stored results. It typically uses iterative loops rather than recursion.

Memoization

- **Definition:** Memoization is a specific technique used in the top-down approach. It involves caching the results of expensive function calls and returning the cached result when the same inputs occur again.

Steps in Memoization

1. **Define the Recursive Function:**
 - Identify the base case(s) and the recursive case(s).
2. **Create a Data Structure:**
 - Use a dictionary, list, or array to store the results of computed subproblems.
3. **Check Cache Before Computation:**

- Before performing any calculations, check if the result is already stored. If so, return it; otherwise, compute it and store the result.

Example

Consider the Fibonacci sequence, where each number is the sum of the two preceding ones.

Recursive Solution without Memoization:

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)
```

Recursive Solution with Memoization:

```
def fib(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]
```

Conclusion

Dynamic programming is a powerful technique used to optimize recursive solutions by storing results of subproblems. Understanding its elements—**optimal substructure and overlapping subproblems**—is essential for applying both memoization and tabulation effectively.

1. Matrix-Chain Multiplication

Problem Statement: Given a sequence of matrices, the task is to determine the most efficient way to multiply these matrices together. The goal is to minimize the total number of scalar multiplications.

Given a sequence of matrices $A_1, A_2, \dots, A_n$, the goal is to determine the most efficient way to multiply these matrices together. The order of multiplication affects the number of scalar multiplications required, and the objective is to minimize this count.

For instance, consider matrices A_1 of dimension $p_0 \times p_1$, A_2 of dimension $p_1 \times p_2$, ..., A_n of dimension $p_{n-1} \times p_n$. The cost of multiplying two matrices A_i and A_j is given by:

$$\text{Cost}(A_i \times A_j) = p_{i-1} \times p_j \times p_i$$

where p_{i-1} , p_i , and p_j are the dimensions of the matrices involved.

Dynamic Programming Approach

1. Define the Dimensions:

- Let the dimension array be p , where A_i has dimensions $p_{i-1} \times p_i$.
- If there are n matrices, the dimension array will have $n+1$ elements.

2. Define the DP Table:

- Create a 2D table $m[i][j]$ where $m[i][j]$ represents the minimum number of scalar multiplications required to multiply matrices A_i through A_j .

3. Base Case:

- When multiplying a single matrix, no multiplications are needed:

$$m[i][i] = 0 \text{ for all } i$$

4. Recurrence Relation:

- For chains of matrices from A_i to A_j calculate:

$$m[i][j] = \min_{i \leq k < j} (m[i][k] + m[k+1][j] + p_{i-1} \times p_k \times p_j)$$

This means you check every possible position k to split the product and take the minimum cost.

5. Filling the DP Table:

- Iterate over increasing lengths of the matrix chain and fill in the DP table based on the recurrence relation.

Matrix-Chain Multiplication(MCM) Pseudocode

MATRIX-CHAIN-ORDER(p, n)

```
1  let  $m[1:n, 1:n]$  and  $s[1:n-1, 2:n]$  be new tables
2  for  $i = 1$  to  $n$                                 // chain length 1
3       $m[i, i] = 0$ 
4  for  $l = 2$  to  $n$                                 //  $l$  is the chain length
5      for  $i = 1$  to  $n - l + 1$                     // chain begins at  $A_i$ 
6           $j = i + l - 1$                         // chain ends at  $A_j$ 
7           $m[i, j] = \infty$ 
8          for  $k = i$  to  $j - 1$                     // try  $A_{i:k}A_{k+1:j}$ 
9               $q = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
10             if  $q < m[i, j]$ 
11                  $m[i, j] = q$                 // remember this cost
12                  $s[i, j] = k$                 // remember this index
13  return  $m$  and  $s$ 
```

```

PRINT-OPTIMAL-PARENS( $s, i, j$ )
1  if  $i == j$ 
2      print " $A$ " $i$ 
3  else print "("
4      PRINT-OPTIMAL-PARENS( $s, i, s[i, j]$ )
5      PRINT-OPTIMAL-PARENS( $s, s[i, j] + 1, j$ )
6      print ")"

```

Example

Let's illustrate this with an example:

- Consider three matrices with the following dimensions:
 - $A_1 : 10 \times 20$
 - $A_2 : 20 \times 30$
 - $A_3 : 30 \times 40$

The dimension array p will be:

$p = [10, 20, 30, 40]$

Here, $n = 3$

Step 1: Initialize the DP Table

Create a table m with dimensions $(n+1) \times (n+1)$:

```

m = [ [0, 0, 0, 0],
      [0, 0, 0, 0],
      [0, 0, 0, 0],
      [0, 0, 0, 0] ]

```

Step 2: Set Base Cases:

- Set $m[i][i] = 0$ for all i :

```

m = [[0, 0, 0, 0],
     [0, 0, 0, 0],
     [0, 0, 0, 0],
     [0, 0, 0, 0]]

```

Step 2: Fill the Table

- For length 2 (i.e., chains of 2 matrices):**

- $m[1][2]$:
 - Split between A_1 and A_2
 $m[1][2] = m[1][1] + m[2][2] + p_0 \times p_1 \times p_2 = 0 + 0 + 10 \times 20 \times 30 = 6000$

- $m[2][3]$:
 - Split between A_2 and A_3 :

$$m[2][3] = m[2][2] + m[3][3] + p_1 \times p_2 \times p_3 = 0 + 0 + 20 \times 30 \times 40 = 24000$$

Updated table:

```
m = [ [0, 0, 0, 0],
       [0, 0, 6000, 0],
       [0, 0, 0, 24000],
       [0, 0, 0, 0] ]
```

2. For length 3 (i.e., the entire chain):

- $m[1][3]$:
 - Split between A_1 and $(A_2 \times A_3)$:

$$m[1][3] = m[1][1] + m[2][3] + p_0 \times p_1 \times p_3 = 0 + 24000 + 10 \times 20 \times 40 = 28000$$

- Split between $(A_1 \times A_2)$ and A_3 :

$$m[1][3] = m[1][2] + m[3][3] + p_0 \times p_2 \times p_3 = 6000 + 0 + 10 \times 30 \times 40 = 6000 + 12000 = 18000$$

The minimum is 18000.

```
m = [ [0, 0, 0, 0],
       [0, 0, 6000, 18000],
       [0, 0, 0, 24000],
       [0, 0, 0, 0] ]
```

Final Result

The minimum number of scalar multiplications required to multiply matrices A_1, A_2 and A_3 is $m[1][3] = 18000$.

Complexity Analysis

- **Time Complexity:** $O(n^3)$, where n is the number of matrices. This is due to the nested loops that fill in the DP table.
- **Space Complexity:** $O(n^2)$, for the DP table.

Conclusion

The Matrix-Chain Multiplication problem effectively illustrates the use of dynamic programming to minimize computational costs by intelligently determining the order of matrix multiplication. The methodology ensures that we avoid unnecessary calculations and achieve optimal performance.

Longest Common Subsequence (LCS)

Problem Definition

Given two sequences X and Y, the Longest Common Subsequence is the longest subsequence that appears in both sequences in the same order, but not necessarily contiguously. For example, for the sequences:

- X="ABCBDA B"
- Y="BDCAB"

The LCS is "BDAB" or "BCAB", both of length 4.

Dynamic Programming Approach

1. Define the DP Table:

- Let $L[i][j]$ be the length of the LCS of the sequences $X[1...i]$ and $Y[1...j]$.

2. Base Cases:

- If either sequence is empty, the LCS length is 0:

$$\begin{aligned} L[i][0] &= 0 \quad \text{for all } i \\ L[0][j] &= 0 \quad \text{for all } j \end{aligned}$$

3. Recurrence Relation:

- If the characters match:

$$L[i][j] = L[i-1][j-1] + 1$$

- If the characters do not match:

$$L[i][j] = \max(L[i-1][j], L[i][j-1])$$

4. Filling the DP Table:

- Iterate through the lengths of both sequences and fill the table based on the above relations.

Pseudocode

Here's the pseudocode for the LCS algorithm:

```
LCS-LENGTH( $X, Y, m, n$ )
1  let  $b[1:m, 1:n]$  and  $c[0:m, 0:n]$  be new tables
2  for  $i = 1$  to  $m$ 
3       $c[i, 0] = 0$ 
4  for  $j = 0$  to  $n$ 
5       $c[0, j] = 0$ 
6  for  $i = 1$  to  $m$           // compute table entries in row-major order
7      for  $j = 1$  to  $n$ 
8          if  $x_i == y_j$ 
9               $c[i, j] = c[i - 1, j - 1] + 1$ 
10              $b[i, j] = "\diagdown"$ 
11         elseif  $c[i - 1, j] \geq c[i, j - 1]$ 
12              $c[i, j] = c[i - 1, j]$ 
13              $b[i, j] = "\uparrow"$ 
14         else  $c[i, j] = c[i, j - 1]$ 
15              $b[i, j] = "\leftarrow"$ 
16  return  $c$  and  $b$ 

PRINT-LCS( $b, X, i, j$ )
1  if  $i == 0$  or  $j == 0$ 
2      return          // the LCS has length 0
3  if  $b[i, j] == "\diagdown"$ 
4      PRINT-LCS( $b, X, i - 1, j - 1$ )
5      print  $x_i$       // same as  $y_j$ 
6  elseif  $b[i, j] == "\uparrow"$ 
7      PRINT-LCS( $b, X, i - 1, j$ )
8  else PRINT-LCS( $b, X, i, j - 1$ )
```

```
function LCS(X, Y):
    m = length(X)
    n = length(Y)
    let L be a 2D array of dimensions (m+1) x (n+1)

    // Initialize the base cases
    for i = 0 to m do
        L[i][0] = 0
    for j = 0 to n do
        L[0][j] = 0

    // Fill the table L
    for i = 1 to m do
        for j = 1 to n do
            if X[i-1] == Y[j-1] then
                L[i][j] = L[i-1][j-1] + 1
            else
                L[i][j] = max(L[i-1][j], L[i][j-1])

    return L[m][n] // The length of LCS
```

Example

Let's illustrate the LCS algorithm with an example using the sequences $X = \text{"ABCB DAB"}$ and $Y = \text{"BDCAB"}$

1. Initialization:

- Lengths: $m=7$ $n=5$.
- Create a DP table L with dimensions $(m+1) \times (n+1)$:

```
L = [ [0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0] ]
```

2. Filling the DP Table:

- For $i=1$ ($X[0] = \text{'A'}$):**
 - $j=1$ ($Y[0] = \text{'B'}$): No match $\rightarrow L[1][1]=0$
 - $j=2$ ($Y[1] = \text{'D'}$): No match $\rightarrow L[1][2]=0$
 - $j=3$ ($Y[2] = \text{'C'}$): No match $\rightarrow L[1][3]=0$
 - $j=4$ ($Y[3] = \text{'A'}$): Match $\rightarrow L[1][4]=1$
 - $j=5$ ($Y[4] = \text{'B'}$): No match $\rightarrow L[1][5]=1$
- For $i=2$ ($X[1] = \text{'B'}$):**
 - $j=1$: Match $\rightarrow L[2][1]=1$
 - $j=2$: No match $\rightarrow L[2][2]=1$
 - $j=3$: No match $\rightarrow L[2][3]=1$
 - $j=4$: Match $\rightarrow L[2][4]=2$
 - $j=5$: Match $\rightarrow L[2][5]=2$

Continuing this process for all i and j , we fill in the table:

```
L = [ [0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 1, 1],
       [0, 1, 1, 1, 2, 2],
       [0, 1, 1, 2, 2, 2],
       [0, 1, 2, 2, 2, 3],
       [0, 1, 2, 2, 3, 3],
       [0, 1, 2, 3, 3, 3],
       [0, 1, 2, 3, 3, 4] ]
```

3. Final Result:

- The length of the longest common subsequence is found at $L[7][5]=4$.

Complexity Analysis

- **Time Complexity:** $O(m \times n)$, where m and n are the lengths of the two sequences.
- **Space Complexity:** $O(m \times n)$ for the DP table. This can be optimized to $O(\min(m, n))$ using two arrays.

Conclusion

The Longest Common Subsequence problem is a classic example of dynamic programming, providing a way to find the longest subsequence shared by two sequences efficiently.